

FAST- NUCES
Object-Oriented Programming
Fall Semester 2025



Project Report

Project Title: Galaxy Wars - The Object-Oriented Space Battle

Submission Date: December 9, 2025

Project Type: Group Project (2 Members)

Members:

Rameen Fatima 24i-0782

Aizah Atif 24i-0840

Table of Contents

1. Overview and Motivation	4
1.1 Project Overview	4
1.2 Motivation	4
2. Game Features and Mechanics	4
2.1 Core Gameplay	4
2.2 Enemies and Obstacles	4
2.3 Resources and Power-Ups	5
2.4 Level System	6
2.5 Scoring System	7
2.6 Health System	7
3. Class Architecture and Design	7
3.1 Total Classes Implemented	8
3.2 Exception Classes (7 classes)	8
3.3 Manager Classes (8 classes)	8
3.4 Game Object Classes (11 classes)	8
3.5 UI and Screen Classes (8 classes)	9
3.6 Utility Classes (4 classes)	9
3.7 Data Structure	9
3.8 Class Relationships	9
4. Object-Oriented Programming Concepts Implementation	10
4.1 Classes and Objects	10
4.2 Encapsulation	10
4.3 Inheritance	11
4.4 Polymorphism	12
4.5 Abstraction	12
4.6 Friend Functions	12
4.7 Static Members	12
4.8 Const Correctness	13
4.9 Operator Overloading	13
4.10 Composition and Aggregation	14
4.11 Copy Constructor and Deep Copy	14
4.12 Virtual Destructor	15
4.13 Dynamic Memory and Pointers	15
4.14 Function Overloading	16
4.15 Inline Functions (through helper functions)	16
5. File Handling and Exception Handling	16
5.1 Text File Handling - High Scores	16
5.2 Text File Handling - Game Saves	17
5.3 Exception Handling	18

6. GUI and Menu System	20
6.1 Main Menu	20
6.2 Pause Menu	20
6.3 Save/Load Screen	21
6.4 High Scores Screen	21
6.5 Game Over Screen	21
6.6 In-Game HUD	22
6.7 Level Transition Screen	23
6.8 Sound System	23
7. Gameplay Screenshots	23
7.1 Main Menu	23
7.2 Level 1 - Easy Mode	24
7.3 Level 2 - Medium Mode	25
7.4 Level 3 - Hard Mode	26
7.5 Level 4 - Boss Battle	27
7.6 Power-Up Collection	28
Screenshots show different power-up effects:	28
7.7 Resource Collection	30
7.8 Pause Menu	31
7.9 Save/Load Screen	31
7.10 High Scores Screen	32
7.11 Game Over Screen - Victory	32
7.12 Game Over Screen - Defeat	33
7.13 Level Transition	34
8. Challenges Faced and Solutions	35
8.1 Challenge: Enemy Spawning Too Close Together	35
8.2 Challenge: Managing Multiple Power-Up States	36
8.3 Challenge: Slow Motion Affecting All Game Objects	36
8.4 Challenge: Boss Enemy Teleportation	36
8.5 Challenge: Deep Copying Game State	37
8.6 Challenge: Resource Milestone Bonuses	37
9.UML Diagram	40
10. Conclusion	40

1. Overview and Motivation

1.1 Project Overview

Galaxy Wars is a 2D space survival game built using C++ and SFML. The player controls a spaceship that must survive waves of enemies and asteroids while collecting resources and power-ups. The game tests reflexes, strategy, and decision-making skills as difficulty increases through different levels.

1.2 Motivation

The motivation behind this project was to create an engaging game that demonstrates all major Object-Oriented Programming concepts learned throughout the semester. We wanted to build something that is both fun to play and technically solid in terms of code structure. We used different creative sprites to have fun with creating something new, also while showcasing core OOP concepts which was the requirement for this project.

2. Game Features and Mechanics

2.1 Core Gameplay

Player Controls:

- Arrow keys to move the spaceship in all four directions
- Spacebar to shoot bullets
- P key to pause/unpause the game

Game Objective:

- Survive enemy waves and avoid asteroids
- Collect resources (red and blue crystals) to increase score
- Collect power ups for temporary advantages
- Defeat 100 enemies to win the game
- Reach score milestones to advance through levels

2.2 Enemies and Obstacles

Asteroids:

- Spawn randomly from the top of the screen
- Different types with varying speeds
- Health points increase with level (1 HP at level 1, 2 HP at level 2, 3 HP at level 3)
- Collision causes 5 damage to player
- Destroying asteroids gives 10 points

Enemy Ships:

1. Easy Enemies (Level 1):

- Basic movement pattern
- 1 hit point
- Slow speed
- Collision causes 10 damage

2. Medium Enemies (Level 2):

- Faster movement
- 2 hit points
- More aggressive behavior
- Collision causes 10 damage

3. Hard Enemies (Level 3):

- Circular movement patterns
- 3 hit points
- Unpredictable paths
- Collision causes 10 damage

4. Boss Enemy (Level 4):

- 10 hit points
- Advanced movement with zigzag patterns
- Shrinks as it takes damage
- Most challenging opponent

Defeating enemies gives 50 points each

2.3 Resources and Power-Ups

Resources:

- **Red Crystals:** Award 15 points each
- **Blue Crystals:** Award 20 points each

- **Milestone Bonuses:** Every 10 total crystals collected gives bonus points equal to the milestone (10 crystals = 10 bonus points, 20 crystals = 20 bonus points, etc.)
- Spawn in waves of 4 crystals at a time

Power-Ups:

1. **Shield (Cyan):**
 - Protects from damage for 15 seconds
 - Changes spaceship color to light blue
 - Animated shield outline appears around ship
2. **Double Fire (Yellow):**
 - Increases fire rate for 10 seconds
 - Changes spaceship color to yellow
 - Bullets become larger and faster
3. **Instant Repair (Green):**
 - Restores health to maximum instantly
 - Green flash effect on collection
 - Critical for survival in later levels
4. **Slow Motion (Magenta):**
 - Slows down all enemies and obstacles for 8 seconds
 - Changes spaceship color to magenta
 - Timer bar shows remaining slow-mo time
 - Gives player time to collect resources or position strategically

Power-ups spawn in waves of 2 at a time

2.4 Level System

Level 1 (Score 0-50):

- 3 asteroids can be active at once
- Easy enemies spawn (2 maximum)
- Asteroid HP: 1, Enemy HP: 1
- Good for learning controls

Level 2 (Score 51-100):

- 5 asteroids can be active
- Medium enemies spawn (3 maximum)
- Asteroid HP: 2, Enemy HP: 2

- Asteroids change to a different texture
- Faster paced gameplay

Level 3 (Score 101-200):

- 7 asteroids can be active
- Hard enemies with circular patterns (4 maximum)
- Asteroid HP: 3, Enemy HP: 3
- Asteroids change to another texture
- Enemies use circular pattern-based movement

Level 4 (Score 201+):

- Boss level - no asteroids
- Boss enemy with 10 HP
- Must reach 300 points to win
- Most challenging level

Each level transition shows a countdown screen with level information and gives the player 5 seconds to prepare.

2.5 Scoring System

- Destroying asteroid: +10 points
- Defeating enemy: +50 points
- Red crystal: +15 points
- Blue crystal: +20 points
- Milestone bonus: +10/20/30/40... points (every 10 total resources)

2.6 Health System

- Player starts with 100 HP
- Maximum health is 100 HP
- Asteroid collision: -5 HP
- Enemy collision: -10 HP
- Health bar shows current/max health visually
- Green portion of bar decreases as health drops
- Game over when health reaches 0

3. Class Architecture and Design

3.1 Total Classes Implemented

The project contains more than 20 classes, exceeding the minimum requirement of 12. Here is a complete list:

3.2 Exception Classes (7 classes)

1. **GameException** - Base exception class for all game errors
2. **FileLoadException** - Thrown when files cannot be opened
3. **TextureLoadException** - Thrown when textures fail to load
4. **FontLoadException** - Thrown when fonts fail to load
5. **InvalidOperationException** - Thrown for invalid operations
6. **NullPointerException** - Thrown when null pointers are accessed
7. **IndexOutOfBoundsException** - Thrown for array index errors
8. **SaveLoadException** - Thrown when save/load operations fail

3.3 Manager Classes (8 classes)

1. **SoundManager** - Handles background music and sound effects
2. **SlowMotionManager** - Manages slow motion effect timing and state
3. **ScoreManager** - Tracks and displays player score
4. **HealthManager** - Manages player health and health bar
5. **EnemyCounter** - Counts defeated enemies and checks win condition
6. **ResourceManager** - Manages collected crystals and milestone bonuses
7. **HighScoresManager** - Saves and loads top 50 high scores
8. **GameSaveManager** - Handles save/load game functionality

3.4 Game Object Classes (11 classes)

1. **Spaceship** - Player controlled spaceship with power-up states
2. **Bullet** - Projectiles fired by the spaceship
3. **Asteroid** - Falling obstacles with varying speeds
4. **EnemyShip** - Base class for all enemy types
5. **EasyEnemy** - Basic enemy (derived from EnemyShip)
6. **MediumEnemy** - Faster enemy (derived from EnemyShip)
7. **HardEnemy** - Pattern-based enemy (derived from EnemyShip)
8. **BossEnemy** - Final boss (derived from HardEnemy)
9. **BaseGameObject** - Base class for resources and power-ups
10. **ResourceObject** - Collectible crystals

11. **PowerUp** - Power-up items with different types

3.5 UI and Screen Classes (8 classes)

1. **Button** - Clickable button for menus
2. **MainMenu** - Main menu screen
3. **SaveLoadScreen** - Save and load game screen
4. **HighScoresScreen** - High scores display screen
5. **PauseMenu** - Pause menu overlay
6. **EnhancedGameOverScreen** - Game over screen with score saving
7. **SlowMoIndicator** - Visual indicator for slow motion
8. **GameTimer** - Timer display for gameplay duration

3.6 Utility Classes (4 classes)

- **LevelManager** - Controls level progression and transitions
- **SpawnPositionTracker** - Prevents objects from spawning too close
- **WaveSpawnManager** - Controls wave-based spawning of resources and power-ups
- **Resource** - Struct for storing crystal counts with operator overloading

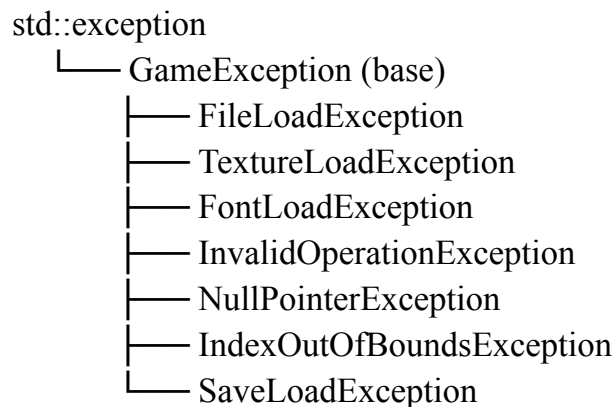
3.7 Data Structure

1. **SaveableState** - Struct that stores complete game state for saving

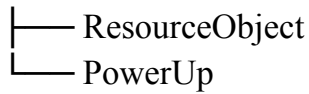
Total: 40 Classes/Structs (far exceeding the 12 class requirement)

3.8 Class Relationships

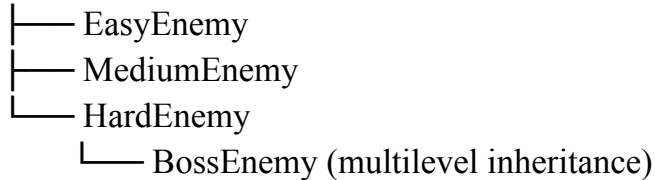
Inheritance Hierarchy:



BaseGameObject



EnemyShip



Composition/Aggregation:

- MainMenu contains multiple Button objects
- SaveLoadScreen contains multiple Button objects
- PauseMenu contains multiple Button objects
- GameSaveManager contains SaveableState pointers
- HighScoresManager contains ScoreEntry array
- Each class with UI contains SFML Text and Font objects

4. Object-Oriented Programming Concepts Implementation

4.1 Classes and Objects

Every game entity is represented as a class with specific attributes and methods. For example, the Spaceship class has position, speed, power-up states, and methods like `handleInput()`, `update()`, and `draw()`.

Example:

```
Spaceship spaceship("C:/proj_oop2/spaceship.png", 0.5f);  
spaceship.handleInput(); // moves based on keyboard  
spaceship.update(deltaTime); // updates power-up timers  
spaceship.draw(window); // renders to screen
```

4.2 Encapsulation

All class data members are private or protected. Public methods provide controlled access. For example, in `HealthManager`, `health` is private and can only be modified through specific methods like `damageFromAsteroid()` or `instantRepair()`.

Example:

```
class HealthManager {
private:
    int health, maxHealth; // encapsulated data
public:
    void damageFromAsteroid() { health -= 5; } // controlled access
    int getHealth() const { return health; } // read-only access
};
```

4.3 Inheritance

Single Inheritance:

- All exception classes inherit from `GameException`
- `EasyEnemy`, `MediumEnemy`, `HardEnemy` inherit from `EnemyShip`
- `ResourceObject` and `PowerUp` inherit from `BaseGameObject`

Multilevel Inheritance:

- `BossEnemy` inherits from `HardEnemy`, which inherits from `EnemyShip`

This allows code reuse. For example, all enemies share common methods like `spawn()`, `update()`, `takeDamage()`, and `draw()` from the base `EnemyShip` class. Each derived class can override these methods to add specific behavior.

Example:

```
class HardEnemy : public EnemyShip {
    // inherits all base functionality
    void update() override {
        // custom circular movement pattern
    }
};
```

4.4 Polymorphism

Dynamic Binding: Enemy arrays use base class pointers to store different enemy types:

```
EnemyShip* currentEnemiesArray[6];
```

At runtime, these pointers can point to EasyEnemy, MediumEnemy, HardEnemy, or BossEnemy objects. When we call methods like update() or draw(), the correct version runs based on the actual object type.

Virtual Functions: Methods like spawn(), update(), and takeDamage() are virtual in base classes and overridden in derived classes.

4.5 Abstraction

EnemyShip is an abstract class with pure virtual spawn() method:

```
virtual void spawn(float windowWidth, SpawnPositionTracker* tracker = nullptr) = 0;
```

This forces all derived enemy classes to implement their own spawn logic. You cannot create an EnemyShip object directly - only objects of derived classes.

4.6 Friend Functions

Three friend functions work with ResourceManager:

- processRedCrystalCollection()
- processBlueCrystalCollection()
- combineResources()

These functions need direct access to private member playerResources to update crystal counts efficiently.

Example:

```
friend void processRedCrystalCollection(ResourceManager& rm, ScoreManager& sm);
```

4.7 Static Members

The GameSaveManager class uses static constants:

```
static const int MAX_SAVE_SLOTS = 3;
```

The Resource struct has operator overloading that creates new Resource objects with static behavior.

4.8 Const Correctness

Many getter methods are marked const to indicate they don't modify the object:

```
int getHealth() const { return health; }  
bool isActive() const { return active; }  
float getSpeedMultiplier() const { return speedMultiplier; }
```

Methods that return error messages override std::exception's what() method with noexcept guarantee:

```
const char* what() const noexcept override {  
    return message.c_str();  
}
```

4.9 Operator Overloading

Resource struct:

```
Resource operator+(const Resource& other) const; // addition  
Resource& operator+=(const Resource& other); // compound assignment  
friend ostream& operator<<(ostream& os, const Resource& r); // output  
friend istream& operator>>(istream& is, Resource& r); // input
```

PowerUp class:

```
bool operator==(const PowerUp& other) const; // equality  
bool operator!=(const PowerUp& other) const; // inequality  
bool operator>(const PowerUp& other) const; // comparison  
bool operator<(const PowerUp& other) const; // comparison
```

```

PowerUp operator+(const PowerUp& other) const; // combination
PowerUp& operator+=(const PowerUp& other); // compound
PowerUp operator*(int multiplier) const; // scaling
friend ostream& operator<<(ostream& os, const PowerUp& pu); // output

```

ResourceObject class:

```

bool operator==(const ResourceObject& other) const;
bool operator!=(const ResourceObject& other) const;

```

These operators make the code more intuitive. For example, combining resources uses simple addition syntax.

4.10 Composition and Aggregation

Composition (strong ownership):

- Button objects are composed inside menu classes (MainMenu, PauseMenu, SaveLoadScreen)
- When a menu is destroyed, its buttons are automatically destroyed
- Each manager class owns its SFML Text and Font objects

Aggregation (weak relationship):

- HighScoresScreen receives a pointer to HighScoresManager but doesn't own it
- SaveLoadScreen receives a pointer to GameSaveManager but doesn't own it
- If the screen is destroyed, the manager still exists

4.11 Copy Constructor and Deep Copy

All classes with dynamic memory implement copy constructors and assignment operators:

Example from GameSaveManager:

```

GameSaveManager(const GameSaveManager& other) {
    // deep copy all data
    for (int i = 0; i < MAX_SAVE_SLOTS; i++) {
        if (other.savedStates[i] != nullptr) {

```

```

        savedStates[i] = new SaveableState(*other.savedStates[i]);
    }
}

```

The `deepCopyState()` helper function ensures that when game states are saved, a complete independent copy is created, not just copying pointers.

4.12 Virtual Destructor

Base classes with virtual functions have virtual destructors:

```

virtual ~EnemyShip() { delete sprite; }
virtual ~BaseGameObject() { delete sprite; }

```

This ensures proper cleanup when deleting derived class objects through base class pointers.

4.13 Dynamic Memory and Pointers

Many classes use dynamic memory with `new/delete`:

- Sprite pointers in game objects
- Button pointers in menu classes
- `SaveableState` pointers in `GameSaveManager`

Proper memory management is implemented with destructors that delete all allocated memory.

Example:

```

~MainMenu() {
    delete startButton;
    delete loadGameButton;
    delete highScoresButton;
    delete exitButton;
}

```

4.14 Function Overloading

The spawn() method is overloaded with different parameters:

```
virtual void spawn(float windowWidth);  
virtual void spawn(float windowWidth, SpawnPositionTracker* tracker);
```

This allows spawning with or without position tracking.

4.15 Inline Functions (through helper functions)

Small utility functions like formatTime() in HighScoresManager act as inline-style helpers:

```
static string formatTime(int seconds) {  
    int minutes = seconds / 60;  
    int secs = seconds % 60;  
    // format as MM:SS  
}
```

5. File Handling and Exception Handling

5.1 Text File Handling - High Scores

HighScoresManager saves top 50 scores to "highscores.txt":

File Format:

```
PlayerName,Score,TimeTaken  
Alice,1250,345  
Bob,980,420  
Charlie,750,380
```

Features:

- Automatically sorts scores in descending order
- Saves player name, score, and time in seconds
- Loads existing scores on game start

- Handles missing files gracefully

Code implementation:

```
void saveScoresToText() {
    ofstream file(textFilename);
    if (!file.is_open()) {
        throw FileLoadException(textFilename);
    }
    file << "PlayerName,Score,TimeTaken" << endl;
    for (int i = 0; i < scoreCount; i++) {
        file << scores[i].playerName << ","
            << scores[i].score << ","
            << scores[i].timeTaken << endl;
    }
}
```

5.2 Text File Handling - Game Saves

GameSaveManager creates separate save files for 3 slots:

- savegame_slot1.txt
- savegame_slot2.txt
- savegame_slot3.txt

SaveableState Structure stores:

- Player position (X, Y)
- Power-up states (shield, double fire timers)
- Score, health, level
- Enemy defeated count
- Crystal counts
- Game time in seconds
- Wave information
- All enemy positions and states (asteroids, easy enemies, medium enemies, hard enemies, boss)
- All resource positions and types
- All power-up positions and types

Features:

- Quick save with F5 (saves to slot 1)
- Quick load with F9 (loads from slot 1)
- Full save/load menu for all 3 slots
- Delete save option for each slot
- Checks if save files exist before loading

5.3 Exception Handling

Custom Exception Classes:

All exceptions inherit from `GameException` which inherits from `std::exception`:

```
class GameException : public exception {
private:
    string message;
    string errorType;
public:
    GameException(const string& type, const string& msg)
        : errorType(type), message "[" + type + "]" + msg {}

    const char* what() const noexcept override {
        return message.c_str();
    }
};
```

Specific Exception Types:

1. **FileLoadException** - When audio files or save files cannot be opened
2. **TextureLoadException** - When image files fail to load
3. **FontLoadException** - When font files fail to load
4. **InvalidOperationException** - For invalid game operations
5. **NullPointerException** - When null pointers are accessed
6. **IndexOutOfBoundsException** - For array access errors
7. **SaveLoadException** - When save or load operations fail

Exception Usage Examples:

In SoundManager constructor:

```
if (!backgroundMusic.openFromFile("C:/proj_oop2/bg.mpeg")) {  
    throw FileLoadException("C:/proj_oop2/bg.mpeg");  
}
```

In texture loading:

```
void loadTexture(Texture& texture, const string& path) {  
    if (!texture.loadFromFile(path)) {  
        throw TextureLoadException(path);  
    }  
}
```

In array access:

```
if (index < 0 || index >= scoreCount) {  
    throw IndexOutOfBoundsException(index, scoreCount);  
}
```

Try Catch Blocks:

All potentially dangerous operations are wrapped in try-catch:

```
try {  
    loadTexture(texture, texturePath);  
} catch (const TextureLoadException& e) {  
    cerr << e.what() << " - Using fallback texture\n";  
    loadTexture(texture, genericPath);  
}
```

The main game loop is also wrapped in a try-catch to handle fatal errors:

```
try {  
    // entire game loop  
} catch (const GameException& e) {
```

```

    cerr << "[FATAL GAME ERROR] " << e.what() << endl;
    return 1;
} catch (const exception& e) {
    cerr << "[FATAL ERROR] " << e.what() << endl;
    return 1;
}

```

6. GUI and Menu System

6.1 Main Menu

Features:

- START - Begin new game
- LOAD PREVIOUS GAME - Go to save/load screen
- HIGH SCORES - View top 50 scores
- EXIT - Close the game

Implementation:

- Uses Button class for each option
- Buttons change color on mouse hover
- Title text in large yellow font
- Clean black background

6.2 Pause Menu

Accessible by pressing P during gameplay

Options:

- RESUME GAME (P) - Continue playing
- SAVE GAME - Go to save menu
- RESTART GAME - Start over from level 1
- QUIT TO MENU - Return to main menu

Features:

- Semi-transparent dark overlay
- All game objects frozen in background

- Timer pauses
- Can be toggled with P key

6.3 Save/Load Screen

Two modes:

- Save mode (from pause menu)
- Load mode (from main menu)

Features:

- 3 save slots displayed
- Shows if slot is used or empty
- DELETE button for each used slot
- Status messages (green for success, red for errors)
- BACK button to return to previous screen

6.4 High Scores Screen

Features:

- Displays top 50 scores in a table format
- Shows rank, player name, score, and time
- Scrollable with mouse wheel
- Sorted in descending order by score
- BACK TO MENU button

Table Layout:

RANK	PLAYER NAME	SCORE	TIME
=====			
1	Rameen	1250	05:45
2	Aizah	980	07:00
3	Zahra	750	06:20

6.5 Game Over Screen

Two versions:

- Victory screen (green) - When player wins
- Game over screen (red) - When player loses

Features:

- Shows final score and level reached
- Name input field
- SAVE SCORE button - Adds score to leaderboard
- MAIN MENU button - Return to main menu
- Text input system for player name

6.6 In-Game HUD

Top Left:

- Score display with large white text

Top Right:

- Health display with text
- Health bar (green fill, red background)
- Shows current/max health

Top Center:

- Enemy counter (enemies defeated / 10)
- Yellow text

Bottom Left:

- Crystal counts (Red and Blue with totals)

Bottom Right:

- Game timer showing elapsed time (MM:SS format)

Bottom Center (when active):

- Slow motion indicator
- Timer bar showing remaining slow-mo time
- Magenta colored

6.7 Level Transition Screen

Displays between levels:

- Large "LEVEL X" title
- Level description and features
- Score requirement for next level
- Enemy/Asteroid HP information
- Countdown timer (5 seconds)
- Progress bar animation
- Semi-transparent dark background

6.8 Sound System

The game includes background music and sound effects managed by the SoundManager class. It plays the space-themed background music during gameplay and different sound effects for actions such as shooting bullets, collecting power-ups, and clicking menu buttons.

These sounds make the game more interactive and give instant feedback to the player. The SoundManager loads audio files at the start of the game and provides simple functions like playMusic() and playPowerUpSound() which are called whenever an event occurs. This keeps sound handling organized and easy to maintain.

7. Gameplay Screenshots

7.1 Main Menu

The main menu shows the game title "GALAXY WARS" in large yellow text. Four blue buttons are arranged vertically: START, LOAD PREVIOUS GAME, HIGH SCORES,

and EXIT. The background is black to match the space theme.



7.2 Level 1 - Easy Mode

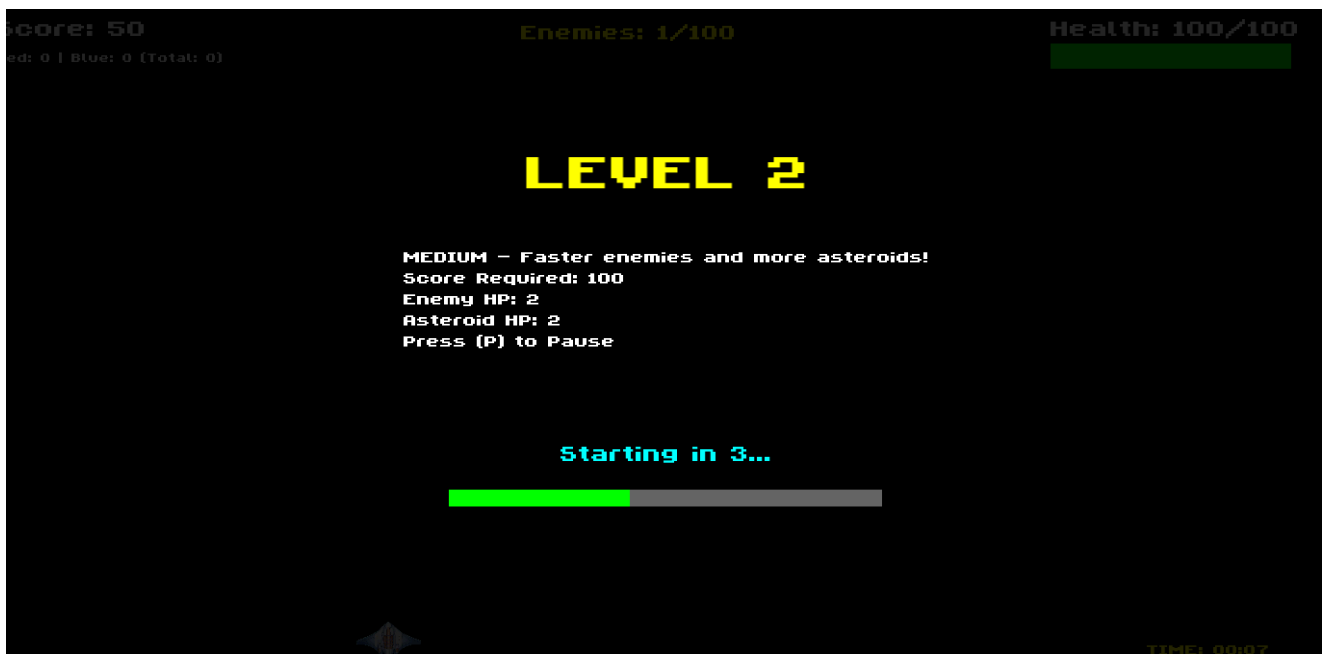
Level 1 features the player's spaceship at the bottom, basic asteroids falling from above, and easy enemy ships. The HUD shows score, health bar, enemy counter, and crystal counts. The spaceship has basic movement and shooting capabilities. Asteroids and enemies have 1 HP each.

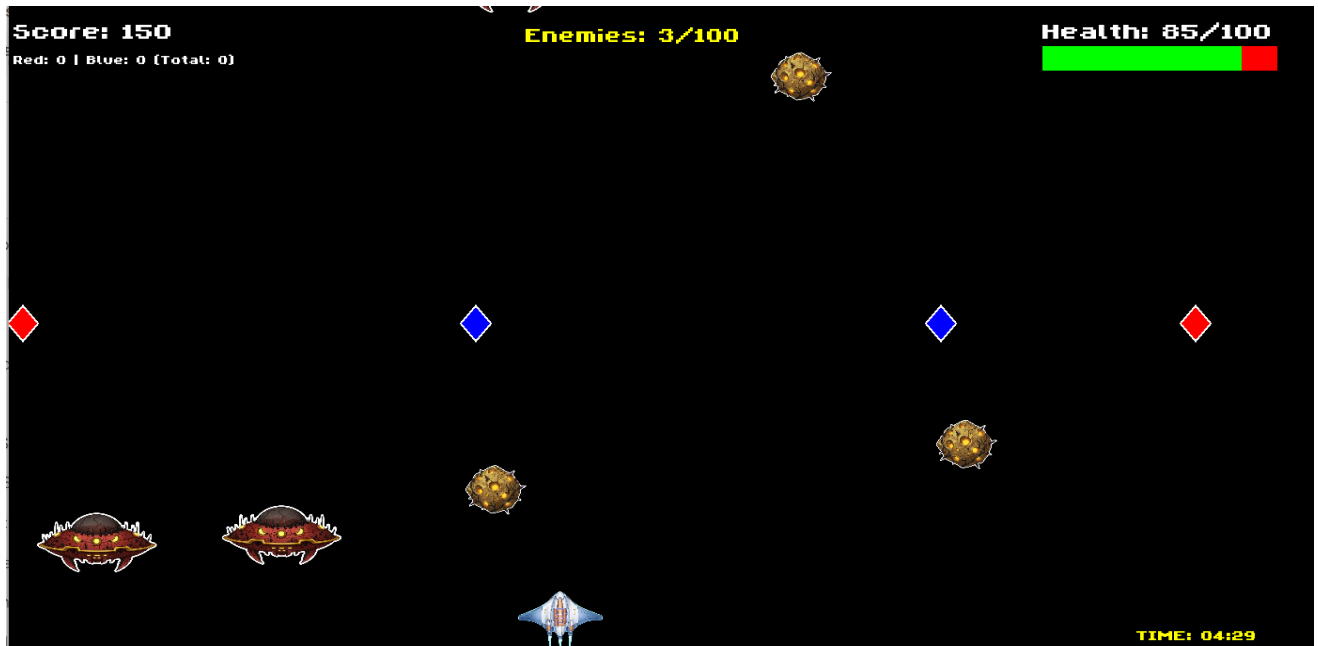




7.3 Level 2 - Medium Mode

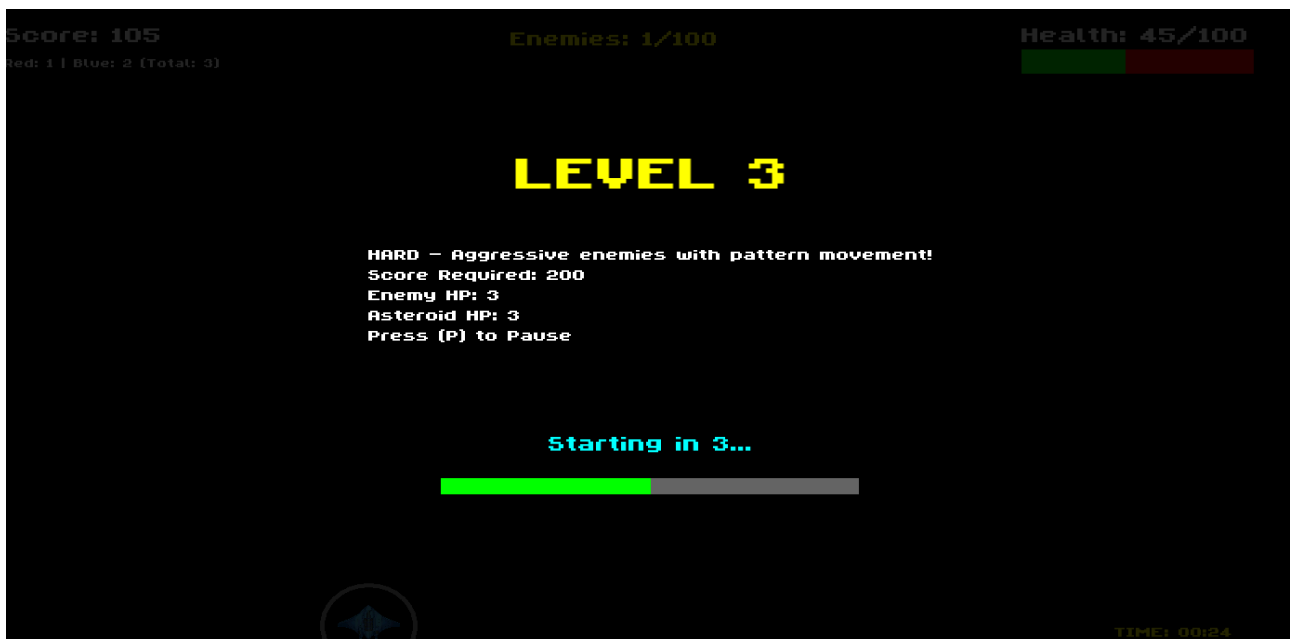
Level 2 increases difficulty with more asteroids (5 active at once) and medium enemies that move faster. The asteroids change to a different texture. Enemies and asteroids now have 2 HP. The screen becomes busier with more objects to avoid.

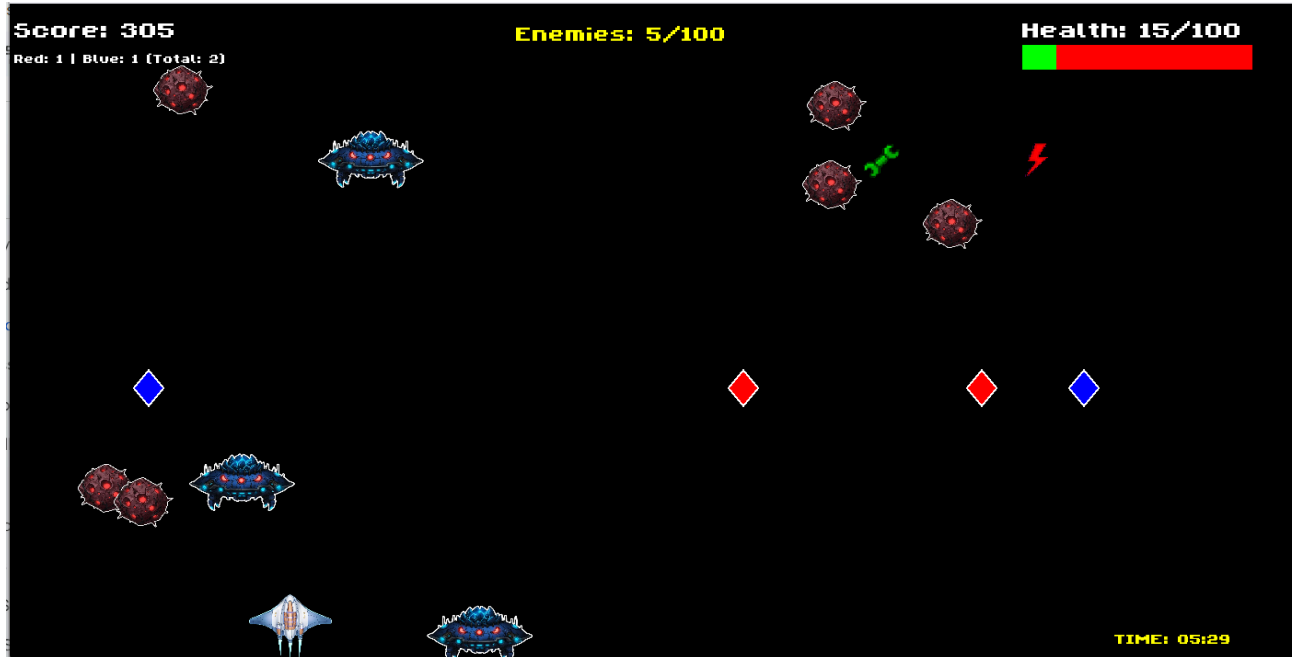




7.4 Level 3 - Hard Mode

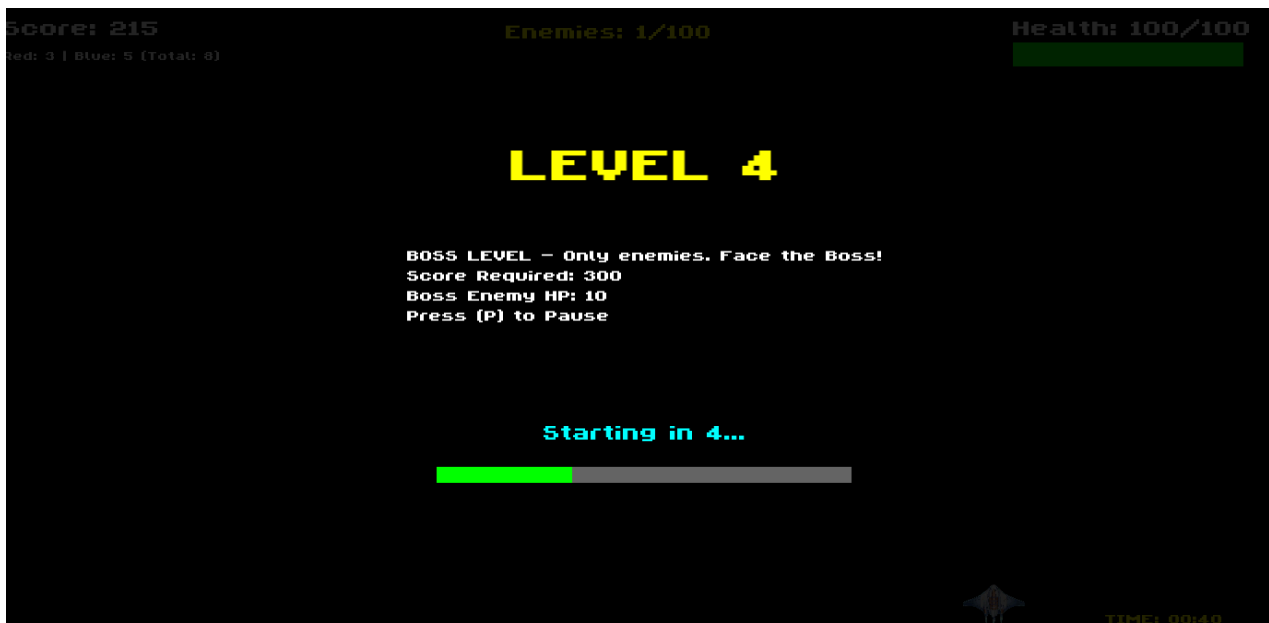
Level 3 shows 7 asteroids on screen simultaneously and hard enemies with circular movement patterns. The asteroid texture changes again. All objects have 3 HP. The gameplay is very intense with pattern-based enemy movement making it harder to predict their paths.





7.5 Level 4 - Boss Battle

The boss level removes all asteroids and features a large boss enemy. The boss has 10 HP and shrinks as it takes damage. It moves in zigzag patterns and can disappear/reappear at random positions. This is the final challenge before winning.





7.6 Power-Up Collection

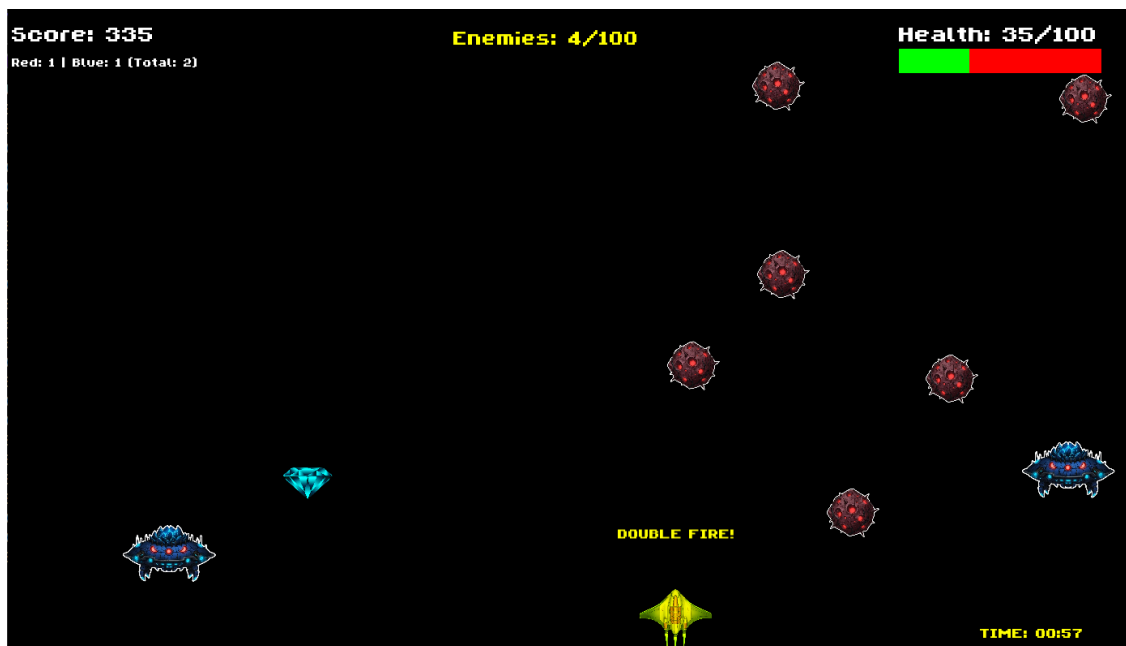
Screenshots show different power-up effects:

- Shield power-up creates a white glowing circle around the spaceship and changes ship color to cyan





- Double Fire power-up changes ship to yellow and displays "DOUBLE FIRE!" text above the ship



- Instant Repair shows green flash effect and "+HEALTH" text



We were unable to capture the green effect on the spot since it only stays for a second. In the screenshot you can see full health.

- Slow Motion changes ship to magenta and displays a timer bar at the bottom showing remaining time



7.7 Resource Collection

Red and blue diamond-shaped crystals fall from the top. The resource counter updates when collected. When total reaches milestones (10, 20, 30...), bonus points are awarded.



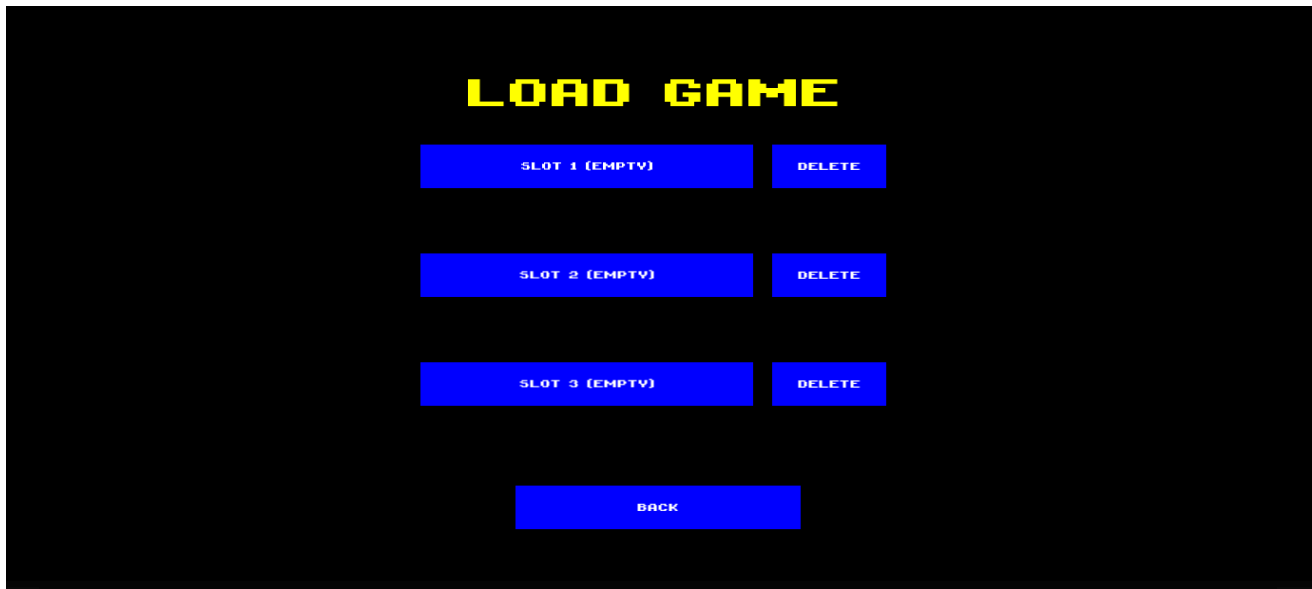
7.8 Pause Menu

The pause menu appears as an overlay with semi-transparent dark background. Game objects are visible but frozen behind it. Four options are displayed: RESUME GAME (P), SAVE GAME, RESTART GAME, and QUIT TO MENU.



7.9 Save/Load Screen

The save/load screen shows three slots vertically arranged. Each slot displays "SLOT X (USED)" or "SLOT X (EMPTY)". Used slots have a DELETE button. A status message appears at the bottom showing success or error messages in green or red.



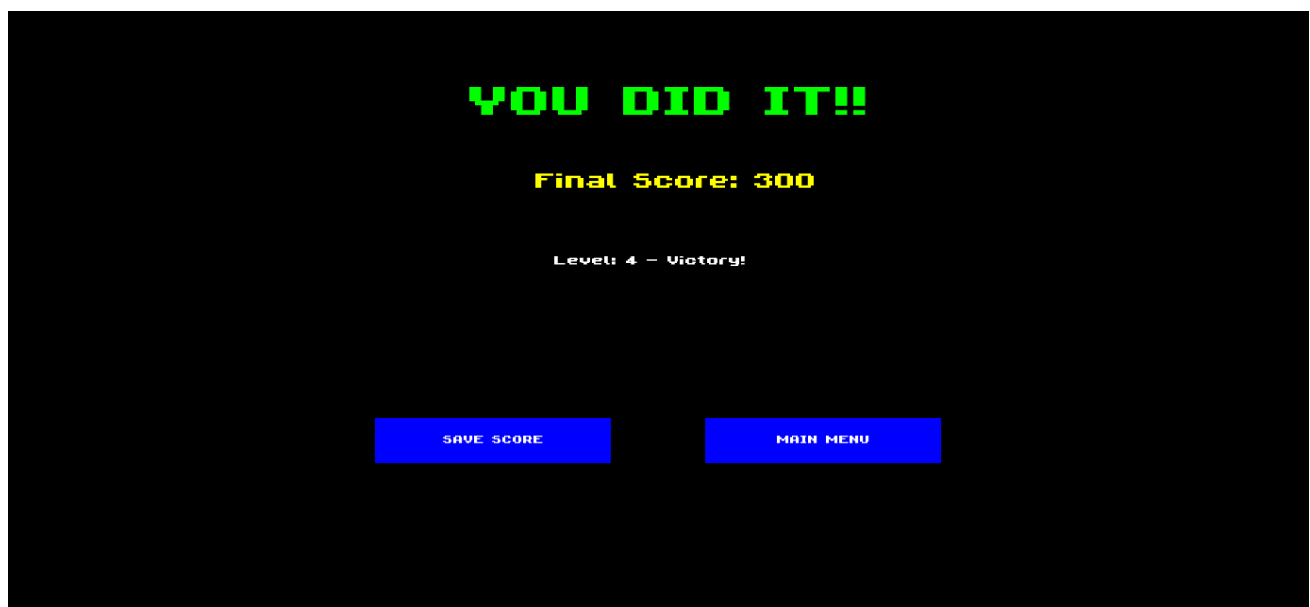
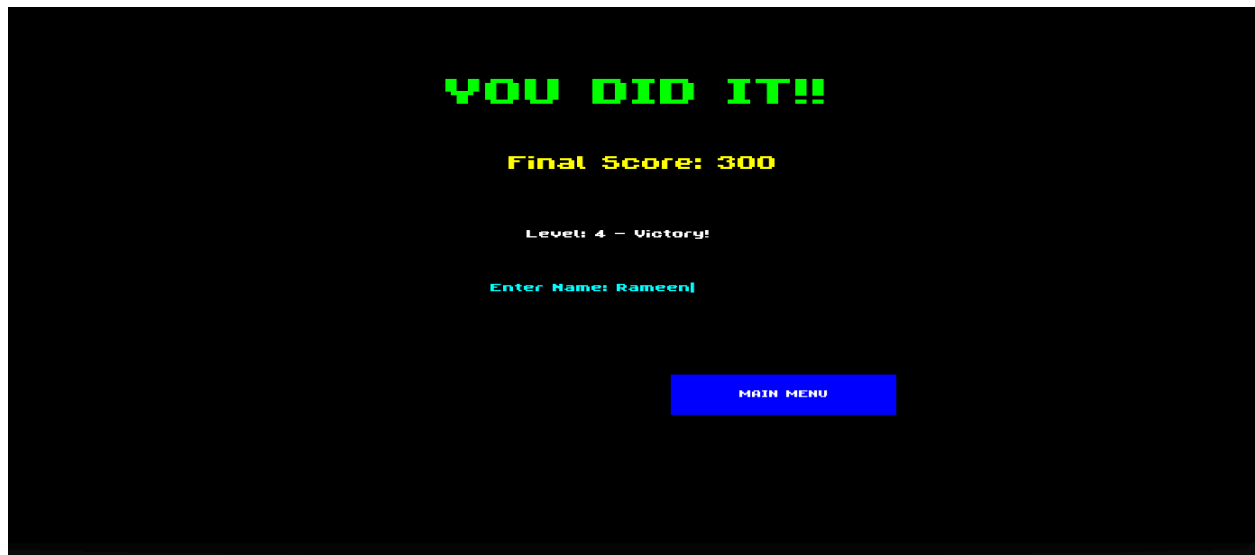
7.10 High Scores Screen

A table displays the top scores with columns for rank, player name, score, and time. The list is scrollable for viewing all 50 entries. A header reads "TOP 50 HIGH SCORES" in yellow.



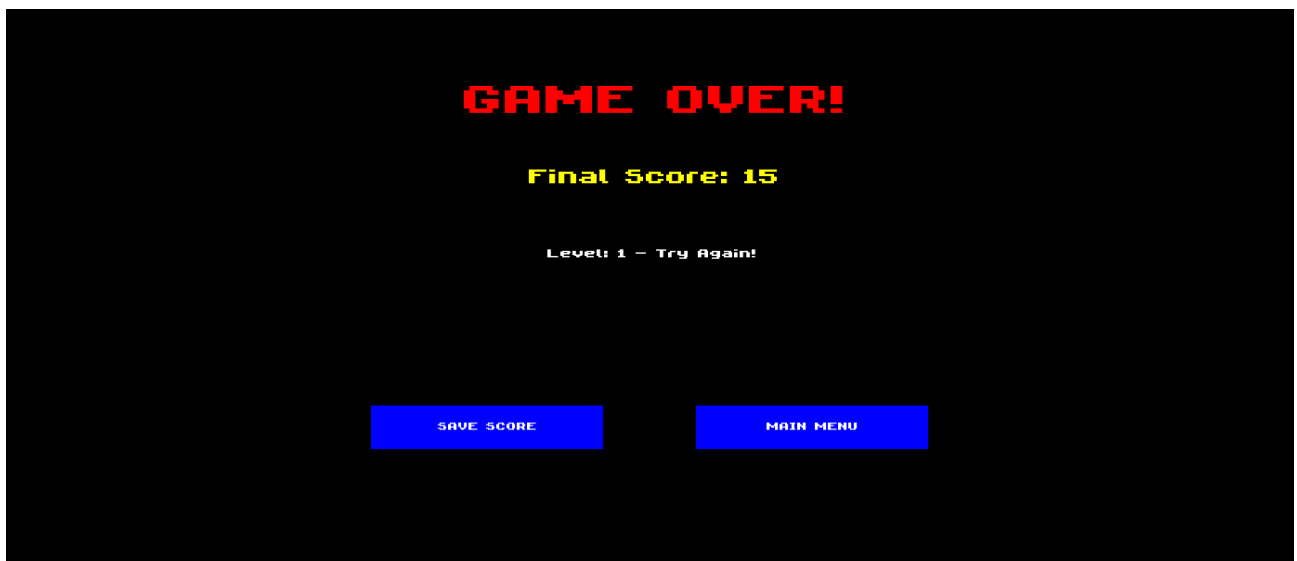
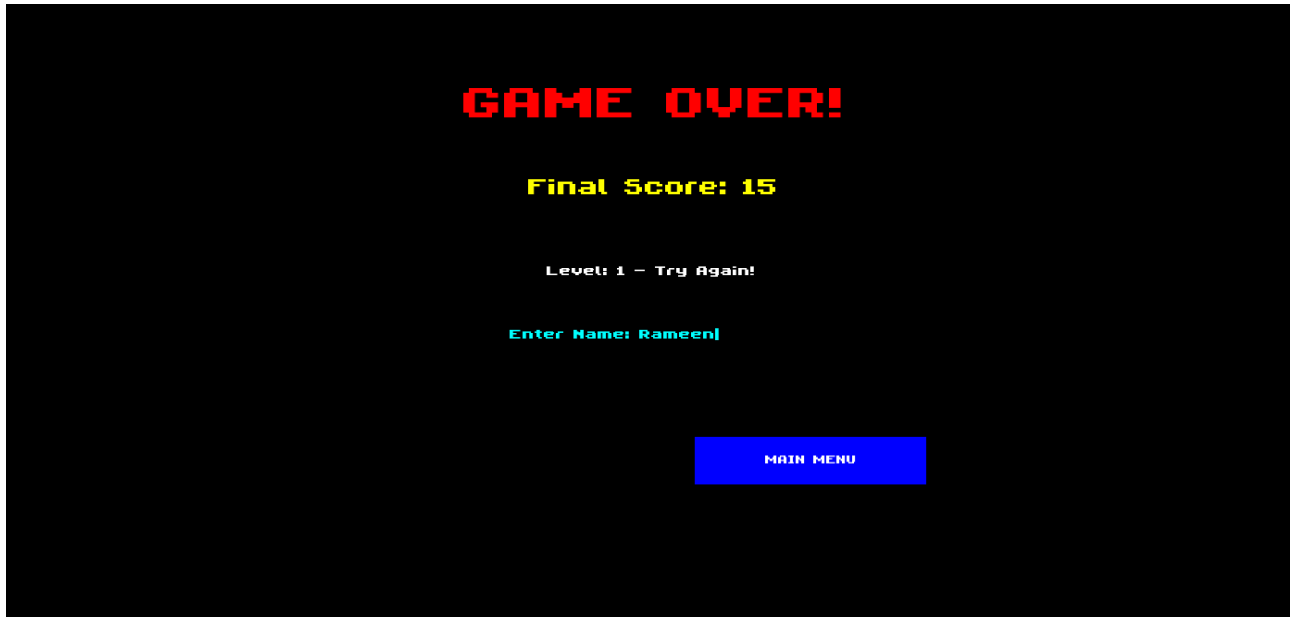
7.11 Game Over Screen - Victory

The victory screen shows "VICTORY!" in green, displays the final score, level reached, and a message "Outstanding Performance!". The player can enter their name and save their score to the leaderboard.



7.12 Game Over Screen - Defeat

The defeat screen shows "GAME OVER!" in red with final score and level. A message reads "Try Again!" and the player can still save their score if it's high enough.



7.13 Level Transition

The transition screen shows a countdown from 5 to 1 with level information, required score, enemy HP, and a progress bar filling from left to right. Instructions like "Press P to Pause" are displayed.



8. Challenges Faced and Solutions

8.1 Challenge: Enemy Spawning Too Close Together

Problem: When multiple enemies spawned at the same time, they sometimes appeared overlapping or very close to each other, making them difficult to separate visually and creating unfair situations for the player.

Solution: We created the `SpawnPositionTracker` class that keeps track of recently used spawn positions. When a new enemy spawns, it checks if the position is too close (within 150 pixels) to any existing spawn points. If it is, a new random position is generated. This ensures enemies are spread out across the screen.

```
bool isPositionOccupied(float x, float tolerance = 150.0f) {  
    for (int i = 0; i < positionCount; i++) {  
        if (x is within tolerance of trackedPositionsX[i])  
            return true;  
    }  
    return false;  
}
```

8.2 Challenge: Managing Multiple Power-Up States

Problem: The spaceship can have multiple power-ups active at once (shield + double fire), and each has its own timer. Managing these states and visual effects was complex.

Solution: We used boolean flags and timer variables in the Spaceship class:

- hasShield, shieldTimer
- hasDoubleFire, fireTimer
- healthFlashTimer

The update() method decrements timers each frame and handles state transitions. The color system prioritizes power-up effects (double fire yellow, shield cyan, normal white) with proper cleanup when effects end.

8.3 Challenge: Slow Motion Affecting All Game Objects

Problem: When implementing slow motion, we needed it to affect enemies, asteroids, bullets, and resources uniformly, but not UI elements or menus.

Solution: We created a global SlowMotionManager (g_slowMotion) that provides a speed multiplier. Every moving object multiplies its speed by this value:

```
float effectiveSpeed = baseSpeed * g_slowMotion.getSpeedMultiplier();
```

When slow motion is active, getSpeedMultiplier() returns 0.3 (30% speed). When inactive, it returns 1.0 (100% speed). This provides centralized control without modifying each class individually.

8.4 Challenge: Boss Enemy Teleportation

Problem: Creating interesting boss behavior with disappearing and reappearing mechanics was tricky. We needed to track visibility state, disappear timer, and reappear cooldown.

Solution: The BossEnemy class tracks:

- isVisible (boolean)
- disappearTimer (counts up while invisible)
- timeSinceLastDisappear (prevents immediate re-disappearing)

The boss stays visible for `reappearCooldown` seconds (8 seconds), then disappears for `disappearDuration` seconds (8 seconds). During invisibility, it cannot be hit and doesn't render. When reappearing, it teleports to a random position using the `teleportToRandomPosition()` method.

8.5 Challenge: Deep Copying Game State

Problem: When saving game state, we needed to create a complete independent copy of all game data. Simple assignment would only copy pointers, causing issues when the original data changes.

Solution: We implemented proper copy constructors and the `deepCopyState()` helper function:

```
void deepCopyState(SaveableState*& dest, const SaveableState& src) {
    if (dest != nullptr) {
        delete dest; // prevent memory leak
    }
    dest = new SaveableState(src); // create new copy
}
```

The `SaveableState` struct has a proper copy constructor that copies all arrays element by element, not just copying pointers.

8.6 Challenge: Resource Milestone Bonuses

Problem: We wanted to reward players for collecting resources with bonus points at milestones (every 10 total resources), but tracking this without double-counting was difficult.

Solution: `ResourceManager` stores `lastMilestone` variable:

```
int currentMilestone = (total / 10) * 10;
if (currentMilestone > lastMilestone && currentMilestone >= 10) {
    int bonusScore = currentMilestone;
    lastMilestone = currentMilestone;
    return bonusScore;
}
```

This ensures bonuses are only given once per milestone. When player collects their 10th resource, they get 10 bonus points. At 20 total, they get 20 bonus points, etc.

8.7 Challenge: Wave-Based Spawning

Problem:

Resources and power-ups needed to spawn in waves rather than all at once. After a wave is collected, the next wave should appear after a delay.

Solution:

WaveSpawnManager class tracks wave number, timer, and spawn status:

Wave 1: Spawn items 0-3

Wave 2: Spawn items 4-7

After delay, check if current wave collected

If yes, move to next wave

Resources spawn in two waves of 4 items each with 4-second delay. Power-ups spawn in two waves of 2 items each with 8-second delay. This creates a steady flow of collectibles without overwhelming the screen.

8.8 Challenge: Level Transition Countdown

Problem:

When moving to a new level, we wanted to give players time to read level information and prepare, not just instantly switch.

Solution:

LevelManager has a transition system:

- showTransition flag
- transitionTimer (counts down from 5 seconds)
- During transition, game logic pauses but rendering continues
- Countdown and progress bar shown on screen
- When timer reaches 0, transition ends and gameplay resumes

This provides a smooth, informative transition between levels.

8.9 Challenge: Exception Handling Throughout Code

Problem:

Many operations can fail (file loading, texture loading, memory allocation) and we needed consistent error handling.

Solution:

We created a hierarchy of exception classes all inheriting from `GameException`. Every risky operation is wrapped in try-catch:

```
try {
    loadTexture(texture, path);
} catch (const TextureLoadException& e) {
    cerr << e.what() << " - Using fallback\n";
    // load generic texture instead
}
```

This prevents crashes and provides helpful error messages. The `main()` function has a final catch-all that handles any unhandled exceptions.

8.10 Challenge: Operator Overloading for Resources

Problem:

We wanted intuitive syntax for combining resources and comparing them.

Solution:

Resource struct implements:

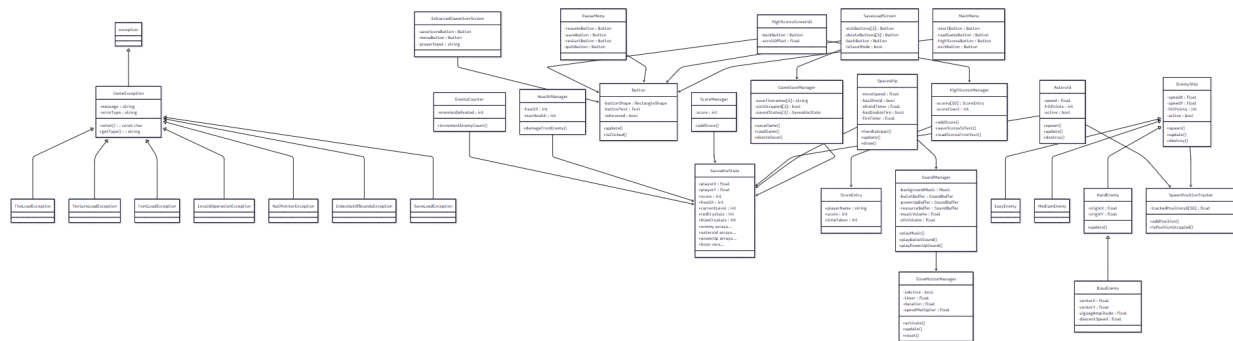
- `operator+` for adding crystals together
- `operator+=` for compound addition
- `operator<<` for output formatting
- `operator>>` for input reading

This allows natural code like:

```
totalResources = resources1 + resources2;
cout << resources; // displays formatted output
```

PowerUp class also overloads comparison operators (`==`, `!=`, `>`, `<`) and arithmetic operators (`+`, `+=`, `*`) for power level calculations.

9.UML Diagram



10. Conclusion

Galaxy Wars successfully demonstrates all major Object-Oriented Programming concepts taught in the course. The game features solid class architecture with over 40 classes, proper inheritance hierarchies, polymorphism through virtual functions, and extensive use of operator overloading.

The gameplay is engaging with four distinct levels, multiple enemy types including a challenging boss, various power-ups, and resource collection mechanics with milestone bonuses. The complete save/load system with three slots allows players to preserve progress, while the high scores system tracks top 50 performances.

Exception handling is implemented throughout the codebase, protecting against file errors, texture loading failures, and invalid operations. File handling manages both high scores (text file) and game saves (text files), with proper error checking and data validation.

The GUI system provides clean, functional menus for all game states: main menu, pause menu, save/load screen, high scores screen, and game over screen. The HUD displays all relevant information clearly during gameplay.

Technical challenges like wave-based spawning, slow motion implementation, boss teleportation mechanics, and deep copying of game state were successfully solved through careful design and testing.

The project not only meets but exceeds the requirements with bonus features including background music, sound effects, boss enemy with unique patterns, dynamic color changes for power-ups, and a full save/load system.

This project represents a complete understanding of object-oriented design principles and their practical application in game development. The code is well-structured, maintainable, and extensible for future enhancements.